

LAPORAN PRAKTIKUM

Modul 10: Javascript Lanjutan



Oleh:

FARIZA NOVIANTI

707012500009

D4 SIKC 49-04

Mata kuliah: Pemrograman Web

**PROGRAM STUDI S.Ter SISTEM INFORMASI KOTA CERDAS
FAKULTAS ILMU TERAPAN
UNIVERSITAS TELKOM
BANDUNG
2025**

DAFTAR ISI

A. Tujuan Praktikum.....	3
B. Alat dan Bahan.....	3
C. Langkah-langkah Praktikum.....	3
D. Hasil dan Dokumentasi Program.....	11
E. Analisis dan Pembahasan.....	12
F. Kesimpulan.....	13
G. Lampiran.....	13

A. Tujuan Praktikum

Setelah mengikuti praktikum ini, mahasiswa diharapkan dapat:

1. Memahami konsep dasar JavaScript sebagai bahasa pemrograman web.
2. Memahami fitur Modern JavaScript (ES6+) dan perannya sebagai fondasi React.
3. Mampu menggunakan sintaks ES6+ secara tepat dan konsisten. Memahami konsep state, immutability, dan functional programming.
4. Mampu membangun aplikasi sederhana berbasis state menggunakan Vanilla JavaScript.

B. Alat dan Bahan

Untuk menjalankan modul ini, diperlukan komputer atau laptop dengan sistem operasi Windows, Linux, atau macOS, minimal prosesor 1 GHz, RAM 2GB (disarankan 4GB atau lebih), serta browser modern seperti Google Chrome, Mozilla Firefox, atau Microsoft Edge.

Perangkat lunak yang digunakan meliputi:

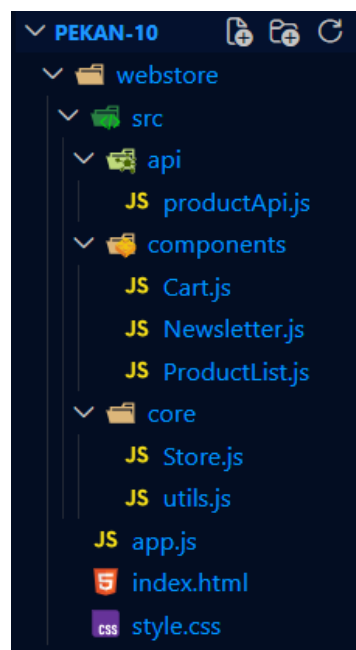
1. Code editor (Visual Studio Code, Sublime Text, atau sejenisnya)
2. Web browser

Bahan yang dibutuhkan:

1. File HTML dasar
2. File JavaScript (.js)

C. Langkah-langkah Praktikum

1. Membuat folder baru bernama webstore di folder Pekan-10 dengan susunan folder di dalamnya sebagai berikut:




```

webstore > style.css > *
38 .breadcrumb a,
39 .toolbar,
40 .toolbar-actions,
41 .feature,
42 .rating-row,
43 .card-footer {
44   display: flex;
45   align-items: center;
46 }
47
48 .btn-primary,
49 .btn-secondary {
50   display: inline-flex;
51   align-items: center;
52   justify-content: center;
53   gap: 8px;
54   border-radius: var(--radius);
55   font-size: 0.875rem;
56   font-weight: 500;
57   cursor: pointer;
58   transition: 0.2s;
59 }
60
61 .svg {
62   width: 16px;
63   height: 16px;
64 }
65
66 .listing-header {
67   margin-bottom: 32px;
68 }
69
70 .page-title {
71   font-family: var(--font-serif);
72   font-size: 1.875rem;
73   font-weight: 700;
74 }

webstore > style.css > *
76 .toolbar {
77   justify-content: space-between;
78   flex-wrap: wrap;
79   margin-top: 16px;
80   gap: 16px;
81 }
82
83 .toolbar-actions {
84   gap: 12px;
85 }
86
87 .breadcrumb {
88   gap: 8px;
89   font-size: 0.875rem;
90   color: var(--text-muted);
91   margin-bottom: 12px;
92 }
93
94 .breadcrumb a {
95   color: var(--text-muted);
96   text-decoration: none;
97   gap: 6px;
98 }
99
100 .breadcrumb a:hover {
101   color: var(--primary);
102 }
103
104 .btn-secondary {
105   background: white;
106   border: 1px solid var(--border-color);
107   color: var(--text-main);
108   padding: 8px 12px;
109 }
110
111 .btn-secondary:hover {
112   background: #f3f4f6;

```

```

webstore > style.css > *
111 .btn-secondary:hover {
112   color: var(--primary);
113 }
114
115
116 .btn-primary {
117   background: var(--primary);
118   color: white;
119   border: none;
120   padding: 10px 20px;
121 }
122
123 .btn-primary:hover {
124   background: var(--primary-hover);
125 }
126
127 .btn-primary svg {
128   width: 20px;
129   height: 20px;
130 }
131
132 .product-grid {
133   display: grid;
134   grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
135   gap: 24px;
136   margin-bottom: 32px;
137 }
138
139 .product-card {
140   background: var(--card-bg);
141   border: 1px solid var(--border-color);
142   border-radius: var(--radius);
143   overflow: hidden;
144   display: flex;
145   flex-direction: column;
146   transition: box-shadow 0.2s;
147   box-shadow: 0 1px 2px 0 rgba(0, 0, 0, 0.05);
148 }

webstore > style.css > *
149
150 .product-card:hover {
151   box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
152 }
153
154 .card-image {
155   height: 220px;
156   padding: 24px;
157   display: flex;
158   justify-content: center;
159   background: white;
160 }
161
162 .card-image img {
163   max-height: 100%;
164   max-width: 100%;
165   object-fit: contain;
166 }
167
168 .card-content {
169   padding: 24px;
170   display: flex;
171   flex-direction: column;
172   flex: 1;
173 }
174
175 .badge {
176   background: var(--primary-light);
177   color: var(--primary);
178   font-size: 0.75rem;
179   font-weight: 600;
180   padding: 2px 10px;
181   border-radius: 4px;
182   width: fit-content;
183   margin-bottom: 12px;
184 }

```

```

webstore > style.css > *
186 .card-title {
187   font-size: 1.125rem;
188   font-weight: 600;
189   color: var(--text-main);
190   text-decoration: none;
191   margin-bottom: 8px;
192   line-height: 1.4;
193   display: -webkit-box;
194   -webkit-line-clamp: 2;
195   line-clamp: 2;
196   -webkit-box-orient: vertical;
197   overflow: hidden;
198 }
199
200 .card-title:hover {
201   text-decoration: underline;
202 }
203
204 .rating-row {
205   gap: 8px;
206   margin-bottom: 16px;
207 }
208
209 .stars {
210   display: flex;
211   color: var(--rating-star);
212 }
213
214 .rating-score {
215   font-size: 0.875rem;
216   font-weight: 600;
217 }
218
219 .rating-count,
220 .feature {
221   font-size: 0.875rem;
222   color: var(--text-muted);
223
225 .features-list {
226   display: flex;
227   gap: 12px;
228   margin-bottom: 20px;
229   list-style: none;
230 }
231
232 .feature {
233   gap: 6px;
234 }
235
236 .card-footer {
237   margin-top: auto;
238   justify-content: space-between;
239 }
240
241 .price {
242   font-size: 1.5rem;
243   font-weight: 800;
244 }
245
246 .cart-section {
247   background: white;
248   border: 1px solid var(--border-color);
249   border-radius: var(--radius);
250   padding: 24px;
251 }
252
253 .cart-section h2 {
254   margin-bottom: 12px;
255   font-family: var(--font-serif);
256 }
257
258 .empty-msg {
259   color: var(--text-muted);
260   font-style: italic;
261 }

```

```

webstore > style.css > *
263 .cart-list {
264   display: flex;
265   flex-direction: column;
266   gap: 16px;
267   margin-bottom: 24px;
268 }
269
270 .cart-item {
271   display: flex;
272   align-items: center;
273   gap: 12px;
274   padding-bottom: 12px;
275   border-bottom: 1px solid var(--border-color);
276 }
277
278 .cart-item img {
279   width: 50px;
280   height: 50px;
281   object-fit: cover;
282   border-radius: 4px;
283   border: 1px solid var(--border-color);
284 }
285
286 .cart-details h4 {
287   font-size: 0.875rem;
288   font-weight: 600;
289   margin-bottom: 4px;
290   display: -webkit-box;
291   -webkit-line-clamp: 1;
292   line-clamp: 1;
293   -webkit-box-orient: vertical;
294   overflow: hidden;
295 }
296
297 .cart-details span {
298   font-size: 0.875rem;
299   color: var(--text-muted);
300
302 .cart-footer-summary {
303   border-top: 2px solid var(--border-color);
304   padding-top: 20px;
305 }
306
307 .total-row {
308   display: flex;
309   justify-content: space-between;
310   align-items: center;
311   margin-bottom: 16px;
312   font-weight: 700;
313   font-size: 1.125rem;
314 }
315
316 .total-price {
317   color: var(--primary);
318 }
319
320 #checkout-btn {
321   width: 100%;
322 }
323
324 .newsletter-section {
325   margin-top: 64px;
326   background: var(--card-bg);
327   border-top: 1px solid var(--border-color);
328   padding: 48px 0;
329   text-align: center;
330 }
331
332 .newsletter-content {
333   max-width: 500px;
334   margin: 0 auto;
335 }
336
337 .newsletter-content h3 {
338   font-family: var(--font-serif);

```

```

webstore > style.css > %
337 .newsletter-content h3 {
338   font-size: 1.5rem;
339   margin-bottom: 8px;
340 }
341
342 .newsletter-content p {
343   color: var(--text-muted);
344   margin-bottom: 24px;
345 }
346
347 .newsletter-content form {
348   display: flex;
349   gap: 8px;
350 }
351
352 .newsletter-content input {
353   flex: 1;
354   padding: 10px;
355   border: 1px solid var(--border-color);
356   border-radius: var(--radius);
357 }
358
359 #app {
360   width: 100%;
361 }
362
363 @media (max-width: 768px) {
364   .card-footer {
365     flex-direction: column;
366     align-items: flex-start;
367     gap: 12px;
368   }
369
370   .newsletter-content form {
371     flex-direction: column;
372   }
373 }
374

```

4. Kemudian, pada file [app.js](#) isi dengan kode program sebagai berikut

```

webstore > JS app.js > render
1 import { fetchProducts } from "../src/api/productApi.js";
2 import { getState, setState } from "../src/core/Store.js";
3 import { ProductList } from "../src/components/ProductList.js";
4 import { Cart } from "../src/components/Cart.js";
5 import { Newsletter } from "../src/components/Newsletter.js";
6
7 function addToCart(product) {
8   const state = getState();
9
10   setState({
11     cart: [...state.cart, product],
12   });
13
14   render();
15 }
16
17 function checkout() {
18   setState({ cart: [] });
19   alert("Thank you for your purchase!");
20   render();
21 }
22
23 function render() {
24   const state = getState();
25
26   ProductList(state.products, addToCart);
27   Cart(state.cart, checkout);
28 }
29
30 async function init() {
31   try {
32     const data = await fetchProducts();
33     setState({ products: data });
34     render();
35     Newsletter();
36   } catch (error) {
37     console.error("Failed to load products:", error);
38   }
39 }
40
41 init();

```

5. Lalu, untuk core/[Store.js](#) isi kode program file tersebut dengan kode program berikut

```
webstore > src > core > JS Store.js > ...
1  import { deepClone } from "../utils.js";
2
3  ∨ let state = {
4      products: [],
5      cart: [],
6  };
7
8  export const getState = () => deepClone(state);
9
10 ∨ export const setState = (newState) => {
11     const nextState = { ...state, ...newState };
12
13     state = nextState;
14
15     console.log("[Store] State updated:", state);
16 };
```

6. Kemudian, pada core/[utils.js](#) isi file dengan kode program berikut

```
webstore > src > core > JS utils.js > ...
1  export function deepClone(obj) {
2      if (obj === null || typeof obj !== "object") {
3          return obj;
4      }
5
6      if (obj instanceof Date) {
7          return new Date(obj.getTime());
8      }
9
10     if (Array.isArray(obj)) {
11         return obj.map((item) => deepClone(item));
12     }
13
14     const clonedObj = {};
15     for (const key in obj) {
16         if (Object.prototype.hasOwnProperty.call(obj, key)) {
17             clonedObj[key] = deepClone(obj[key]);
18         }
19     }
20
21     return clonedObj;
22 }
```

7. Selanjutnya, pada components/[Cart.js](#) isi file dengan kode program berikut

```
webstore > src > components > JS Cart.js > ☑ Cart
1  export function Cart(cart, onCheckout) {
2      const container = document.getElementById("cart");
3      container.innerHTML = "<h2>Your Cart</h2>";
4
5      if (cart.length === 0) {
6          container.innerHTML += "<p class='empty-msg'>Cart is empty.</p>";
7          return;
8      }
9
10     const list = document.createElement("div");
11     list.className = "cart-list";
12
13     cart.map((item) => {
14         const div = document.createElement("div");
15         div.className = "cart-item";
16         div.innerHTML = `
```



```

17     
18     <div class="cart-details">
19       <h4>${item.title}</h4>
20       <span>${item.price}</span>
21     </div>
22   `;
23   list.appendChild(div);
24 });
25
26 container.appendChild(list);
27
28 const total = cart.reduce((acc, item) => acc + (item.price || 0), 0);
29
30 const footer = document.createElement("div");
31 footer.className = "cart-footer-summary";
32 footer.innerHTML = `
33   <div class="total-row">
34     <span>Total:</span>
35     <span class="total-price">${total.toFixed(2)}</span>
36   </div>
37   <button class="btn-primary" id="checkout-btn">Buy Now</button>

```

```

webstore > src > components > JS Cart.js > Cart
1  export function Cart(cart, onCheckout) {
38  `;
39
40  footer.querySelector("#checkout-btn").addEventListener("click", onCheckout);
41  container.appendChild(footer);
42  }

```

8. Lalu, pada components/[Newsletter.js](#) isi file dengan kode program berikut

```

webstore > src > components > JS Newsletter.js > ...
1  export function Newsletter() {
2    const $container = $('<div class="newsletter-section"></div>');
3    const $content = `(
4      <div class="newsletter-content">
5        <h3>Subscribe to our updates</h3>
6        <p>Get the latest tech news and offers.</p>
7        <form id="newsletter-form">
8          <input type="email" placeholder="Enter your email" required />
9          <button type="submit" class="btn-primary">Subscribe</button>
10       </form>
11       <div class="success-message" style="display:none; margin-top: 10px; color: green;">
12         Thanks for subscribing!
13       </div>
14     </div>
15   `);
16
17   $container.append($content);
18   $("#app").append($container);
19
20   $content.find("form").on("submit", function (e) {
21     e.preventDefault();
22     const $form = $(this);
23
24     $form.slideUp(400, function () {
25       $content.find(".success-message").fadeIn();
26     });
27   });
28 }

```

9. Pada components/ProductList.js buat kode program berikut

```

webstore > src > components > JS ProductList.js > ProductList > products.map() callback
1  export function ProductList(products, onAdd) {
2    const container = document.getElementById("product-list");
3    container.innerHTML = "";
4
5    products.map((product) => {
6      const card = document.createElement("div");
7      card.className = "product-card";
8
9      const discount = Math.floor(Math.random() * 20) + 5;
10     const rating = product.rating || { rate: 4.5, count: 100 };
11

```

```

website > src > components > JS ProductList.js > ProductList > products.map() callback
1  export function ProductList(products, onAdd) {
5    products.map((product) => {
37      <div class="card-footer">
38        <span class="price">${product.price}</span>
39        <button class="add-btn btn-primary" type="button">
40          <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24">
41            <path stroke="currentColor" stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 4h1.5L8 16m0 0h8m-8 0a2 2 0 1 0
42            </svg>
43            Add
44          </button>
45        </div>
46      </div>
47    }
48  );
49  card.querySelector(".add-btn").addEventListener("click", function () {
50    onAdd(product);
51  });
52
53  container.appendChild(card);
54
55  });
56
57  function getStarsSVG() {
58    return <svg xmlns="http://www.w3.org/2000/svg" fill="currentColor" viewBox="0 0 20 20">
59      <path d="M9.049 2.927c-.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1 0 0 .95.69h3.462c.969 0 1.371 1.24.588 1.811-2.8 2.034a1 1 0 0 0
60      </svg>;
61  }

```

10. Selanjutnya, di `api/ProductApi.js` isi dengan kode program berikut

```

webstore > src > api > JS productApis > fetchProducts > <function> > setTimeout() callback
1 export const fetchProducts = async () => {
2   return new Promise((resolve) => {
3     setTimeout(() => {
4       resolve([
5         {
6           id: 1,
7           title: "Mechanical Keyboard",
8           price: 150,
9           rating: { rate: 4.8, count: 230 },
10          image: "https://images.unsplash.com/photo-1618384887929-16ec33fab9ef?auto=format&fit=crop&w=800&q=80",
11        },
12        {
13          id: 2,
14          title: "Gaming Mouse",
15          price: 80,
16          rating: { rate: 4.6, count: 150 },
17          image: "https://images.unsplash.com/photo-1527814050087-3793815479db?auto=format&fit=crop&w=800&q=80",
18        },
19        {
20          id: 3,
21          title: "4K Monitor 24\"",
22          price: 300,
23          rating: { rate: 4.9, count: 85 },
24          image: "https://images.unsplash.com/photo-1527443224154-c4a3942d3acf?auto=format&fit=crop&w=800&q=80",
25        },
26        {
27          id: 4,
28          title: "Premium Coffee Beans",
29          price: 25,
30          rating: { rate: 4.7, count: 500 },
31          image: "https://images.unsplash.com/photo-1559056199-641a0ac8b55e?auto=format&fit=crop&w=800&q=80",
32        },
33        {
34          id: 5,
35          title: "Travel Tumbler",
36          price: 35,
37          rating: { rate: 4.5, count: 120 },

```

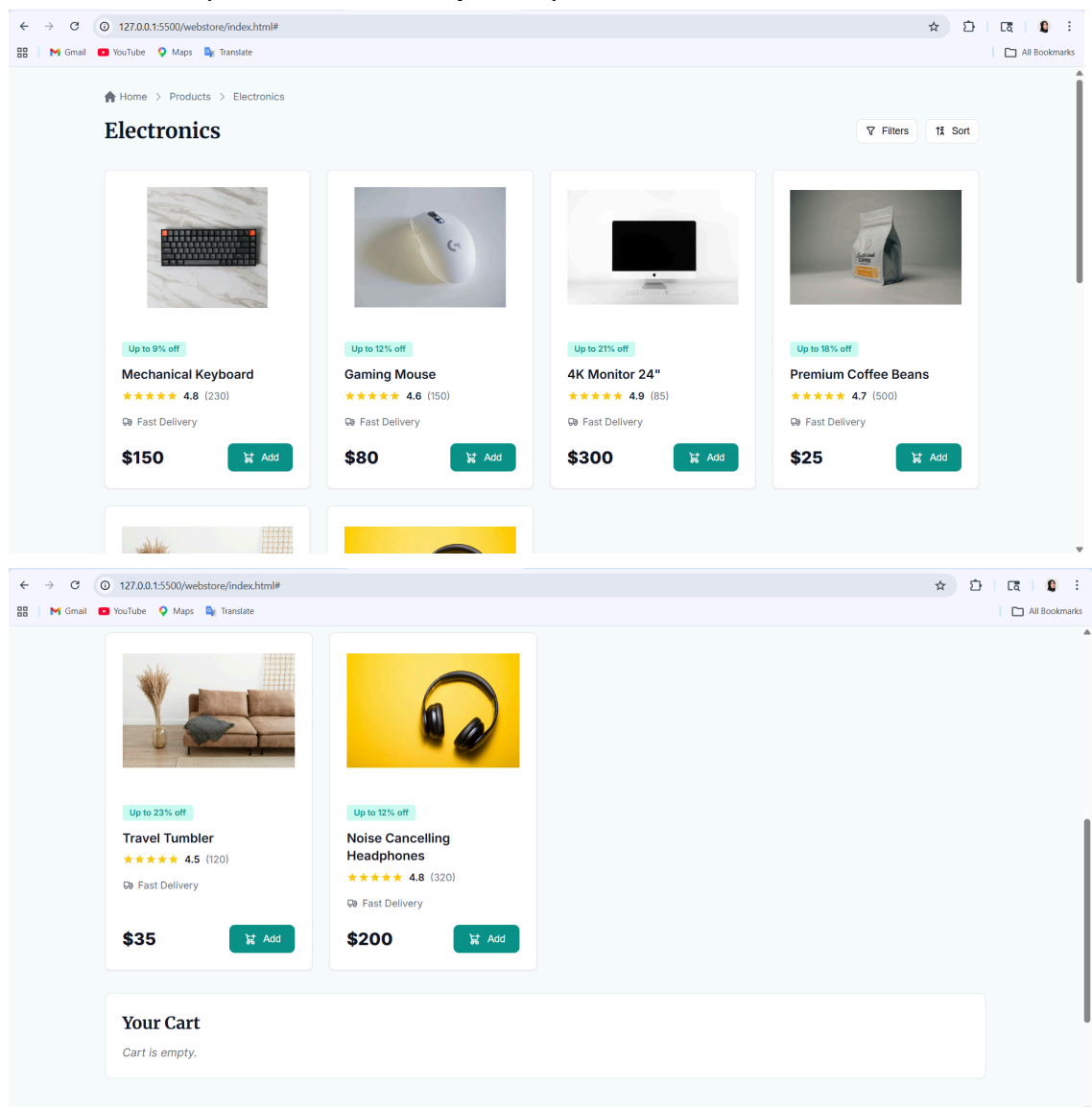
```

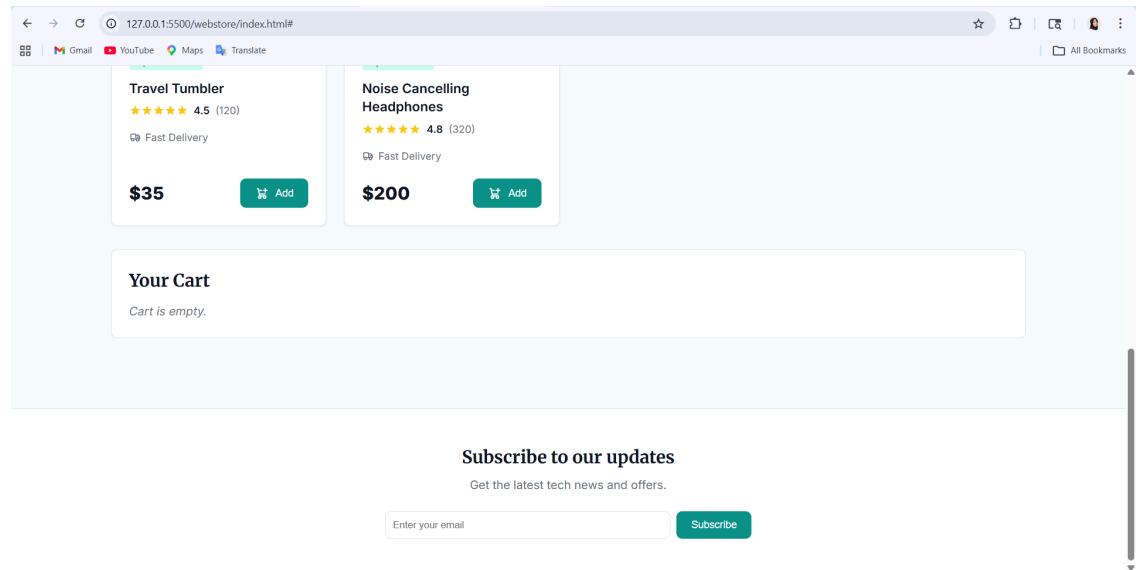
webstore > src > api > JS productApi.js > fetchProducts > <function> > setTimeout() callback
1  export const fetchProducts = async () => {
2    return new Promise((resolve) => {
3      setTimeout(() => {
38         image: "https://images.unsplash.com/photo-1578500494198-246f612d3b3d?auto=format&fit=crop&w=800&q=80",
39       },
40     ),
41     {
42       id: 6,
43       title: "Noise Cancelling Headphones",
44       price: 200,
45       rating: { rate: 4.8, count: 320 },
46       image: "https://images.unsplash.com/photo-1505740420928-5e560c06d30e?auto=format&fit=crop&w=800&q=80",
47     },
48   });
49 });
50 };

```

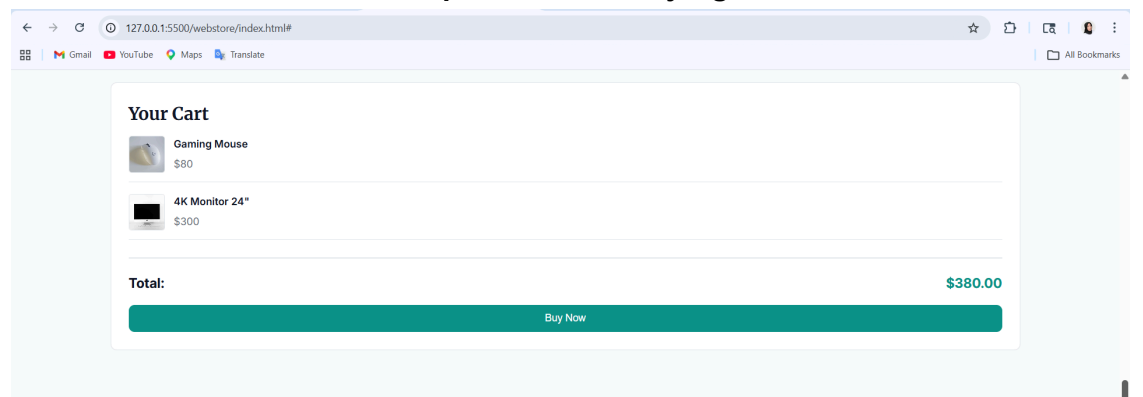
D. Hasil dan Dokumentasi Program

1. Halaman Awal (sebelum membeli produk)

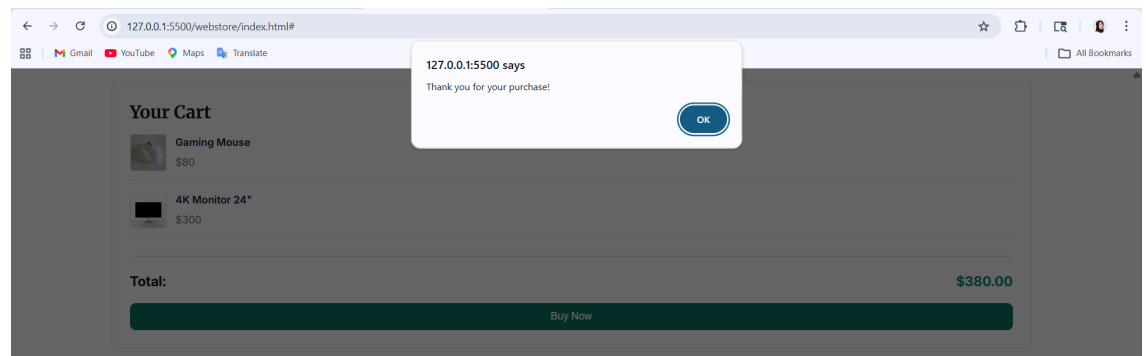




2. Halaman ketika menambahkan produk ke keranjang



3. Respons ketika kita checkout atau membeli produk di keranjang



E. Analisis dan Pembahasan

Praktikum Modul 10 ini membahas penerapan JavaScript modern (ES6+) secara menyeluruh melalui pembuatan sebuah aplikasi Vanilla JS Web Store. Berikut adalah analisis dari setiap konsep yang diterapkan dalam proyek tersebut.

1. Modularisasi dengan ES6 Module (import/export)

Proyek webstore menggunakan sistem modul ES6 dengan keyword `export` dan `import` untuk memisahkan kode ke dalam file-file yang terorganisir, yaitu `app.js`, `Store.js`, `utils.js`, `Cart.js`, `Newsletter.js`, `ProductList.js`, dan `productApi.js`. Pemisahan ini membuat kode lebih mudah dibaca, dipelihara, dan dikembangkan. Setiap file memiliki tanggung jawab yang jelas, sehingga perubahan di satu bagian tidak mengganggu bagian lain. Hal ini merupakan fondasi dari arsitektur berbasis komponen seperti yang digunakan React.

2. State Management dengan Immutability

File `Store.js` mengimplementasikan state management terpusat menggunakan prinsip immutability. State aplikasi (`products` dan `cart`) tidak pernah dimodifikasi langsung, melainkan selalu diganti dengan objek baru menggunakan spread operator (`{ ...state, ...newState }`). Fungsi `getState()` mengembalikan deep clone dari state menggunakan utilitas `deepClone()` di `utils.js`, sehingga komponen tidak dapat secara tidak sengaja mengubah state asli. Pendekatan ini mencegah bug tersembunyi dan memastikan perubahan state terdeteksi dengan benar oleh sistem rendering, menyerupai cara kerja `useState` pada React.

3. Asynchronous JavaScript dan Data Fetching

Fungsi `fetchProducts()` pada `productApi.js` menggunakan `Promise` dan `async/await` untuk mensimulasikan pengambilan data secara asynchronous dengan delay 800ms menggunakan `setTimeout`. Di `app.js`, fungsi `init()` memanggil `fetchProducts()` dengan `async/await` dan menangani error menggunakan `try/catch`, sehingga aplikasi tidak crash jika terjadi kegagalan pengambilan data. Pola ini identik dengan penggunaan `fetch()` yang dikombinasikan dengan `useEffect` pada React, di mana data hasil fetch disimpan ke dalam state dan perubahan state memicu render ulang tampilan.

4. Higher Order Function dan Functional Programming

Komponen `ProductList.js` dan `Cart.js` memanfaatkan higher order function `map()` secara ekstensif untuk mengubah setiap data produk dan item keranjang menjadi elemen HTML secara dinamis. Pendekatan functional ini menghindari perulangan imperatif (`for loop`) dan menghasilkan kode yang lebih ringkas dan mudah dibaca. Fungsi `reduce()` juga digunakan di `Cart.js` untuk menghitung total harga dari seluruh item di keranjang (`cart.reduce((acc, item) => acc + item.price, 0)`). Callback function digunakan saat meneruskan fungsi `addToCart` dan `checkout` sebagai parameter ke komponen anak, menyerupai `props` pada React.

5. Deep Clone dan Penanganan Data Nested

Fungsi `deepClone()` di `utils.js` mengimplementasikan algoritma rekursif untuk menyalin objek hingga ke level terdalam, menangani berbagai tipe data seperti `null`,

Date, Array, dan Object biasa. Ini mengatasi keterbatasan shallow copy yang hanya menyalin referensi pada level pertama. Penggunaan deep clone pada `getState()` memastikan bahwa setiap komponen menerima salinan state yang benar-benar independen, sehingga mutasi tidak disengaja dari luar tidak dapat merusak state global aplikasi.

6. Event Handling dan Integrasi jQuery

Komponen `ProductList.js` dan `Cart.js` menggunakan `addEventListener()` native JavaScript untuk menangani event klik pada tombol “Add” dan “Buy Now”. Sementara itu, komponen `Newsletter.js` secara khusus menggunakan jQuery untuk membangun UI dan menangani event submit formulir. jQuery menyederhanakan manipulasi DOM dengan sintaks `$()`, serta menyediakan efek animasi seperti `.slideUp()` dan `.fadeIn()` yang memperhalus pengalaman pengguna. Perpaduan Vanilla JS dan jQuery dalam satu proyek ini membuktikan bahwa keduanya dapat digunakan secara bersamaan sesuai kebutuhan.

F. Kesimpulan

Berdasarkan praktikum Modul 10 yang telah dilaksanakan, dapat disimpulkan beberapa hal berikut:

1. JavaScript modern (ES6+) menyediakan fitur-fitur penting seperti `let/const`, arrow function, destructuring, spread operator, dan module system yang membuat kode lebih ringkas, aman, dan mudah dikelola dibandingkan JavaScript versi lama (ES5).
2. Konsep immutability sangat penting dalam pengelolaan state aplikasi. Dengan tidak mengubah data secara langsung melainkan selalu membuat salinan baru menggunakan spread operator atau deep clone, alur data menjadi lebih mudah dilacak dan bug tersembunyi dapat dihindari.
3. Functional programming dengan higher order function seperti `map()`, `filter()`, dan `reduce()` memungkinkan pengolahan data array secara deklaratif tanpa mengubah data asli, sehingga kode lebih mudah diprediksi dan diuji.
4. Asynchronous JavaScript menggunakan Promise dan `async/await` memungkinkan pengambilan data dari server tanpa menghentikan eksekusi program utama. Pola ini merupakan fondasi dari data fetching pada aplikasi React menggunakan `useEffect`.
5. Sistem modul ES6 (`export/import`) memungkinkan pemisahan kode ke dalam file-file yang terstruktur dan reusable, yang merupakan dasar dari arsitektur berbasis komponen pada framework modern seperti React.
6. jQuery terbukti dapat diintegrasikan bersama Vanilla JS dalam satu proyek untuk mempermudah manipulasi DOM dan menambahkan efek animasi pada komponen tertentu, seperti yang diterapkan pada komponen Newsletter di proyek webstore ini.

G. Lampiran

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/FarizaNovianti-707012500009>